Lab5 -Reverse Engineering (Test yourself)

Angel Nicolas Cuervo Naranjo

Conceptual: What is the main difference between a disassembler and a debugger? In what situation would you use each one?

1.1 Conceptual Definition

Disassembler

A disassembler is a static analysis tool that converts machine code (binary instructions, i.e., bytes in hexadecimal) back into assembly language readable by a human analyst. It operates on the binary file without executing it, generating a representation of all the instructions that the processor would execute. Examples used in this lab include objdump and Ghidra's disassembly view.

The key characteristic of a disassembler is that it works in a completely offline manner: the binary is never loaded into a processor. For this reason, it poses no risk of accidentally triggering malicious behavior in a file being analyzed. In addition, tools such as Ghidra go one step further by applying a decompiler, reconstructing an approximate version of the original C pseudocode — as seen in Image 23 of the lab, where the pseudocode of the passguess binary was reconstructed.

Debugger

A debugger is a dynamic analysis tool that executes the binary in a controlled environment, allowing the analyst to pause execution at specific points (breakpoints), inspect the state of registers and memory, and follow the program's behavior step by step at runtime. The tool used in this lab was GDB (GNU Debugger).

Unlike the disassembler, the debugger interacts with the program as it actually runs on the processor. This means that memory values, register states, and execution paths are real and not reconstructed — they reflect the true behavior of the code at the exact moment of execution.

1.2 When to Use Each Tool

Use a Disassembler when:

•You need to understand the overall structure of a binary without running it — for example, identifying all functions, imported libraries (e.g., strcmp, scanf), and stored strings. In the lab, running strings passguess | less revealed the hardcoded password directly in the data section.

•The binary is potentially malicious. A disassembler lets you examine malware safely without infecting the system.

•You want to reconstruct pseudocode. As shown in Image 23, Ghidra produced readable C code that made the authentication logic immediately clear: strcmp(user_input, "7Wtyr") with an if/else to print success or failure.

•You need to map the entire Control Flow Graph (CFG). In Part 4 of the lab, Cutter generated a full CFG showing all decision nodes and execution paths without running the binary.

Use a Debugger when:

•You need to confirm that runtime behavior matches the static analysis. In Part 3 of the lab, GDB verified that $rsi actually held the string '7Wtyr' and $rdi held the user input 'hello' at the moment of the strcmp call — confirming what Ghidra had reconstructed statically.

•The binary uses runtime-generated or obfuscated values that are not visible in static analysis. A debugger captures the real decrypted or computed value in memory.

•You want to modify execution flow. Using GDB, you can change the value of $rax after strcmp to force the program into the success branch even with a wrong password, bypassing the authentication check.

•You need to measure performance or trace exact memory allocations at runtime.

Practice: Given an unknown binary, describe your analysis methodology step by step, starting with static tools and progressing to dynamic ones.

Phase 1 – Basic Static Analysis (Initial Reconnaissance)

Objective

Gather the maximum amount of information about the binary without executing it, using low-level command-line tools.

Step 1.1 – Determine the file type

```
$ file passguess
```

This command reveals the binary architecture, whether it is statically or dynamically linked, and — crucially — whether debug symbols have been stripped. In the lab, the output showed: ELF 64-bit LSB executable, x86-64, dynamically linked, not stripped. The fact that it was not stripped meant function names such as main were preserved, significantly simplifying subsequent analysis.

Step 1.2 – Extract readable strings

```
$ strings passguess | less
```

This command dumps all printable character sequences embedded in the binary. The lab identified: the security function __stack_chk_fail (stack canary protection), the input function __isoc23_scanf, the comparison function strcmp (a strong indicator that a password is stored somewhere), user-facing messages such as 'Guess The Pass:', '[OK] Passowrd Found', and '[ERROR] Password is incorrect', and the suspicious strings PTE1, u3UH, and 7Wtyr — the main candidates for the password.

Step 1.3 – Disassemble and identify key functions

```
$ objdump -d passguess | less
```

The disassembly exposed the full assembly listing section by section: the _init and _start startup sequences before main, the stack canary setup at offset 0x117d using %fs:0x28, the scanf call at 0x11ce for reading user input with the %255s format specifier (limiting input to 255 characters), the strcmp call at 0x11e7 comparing input to the stored password, and the conditional branch at 0x11ec — test %eax, %eax followed by jne 0x1201 — that routes execution to the success or error block.

Phase 2 – Advanced Static Analysis (Decompilation with Ghidra)

Objective

Reconstruct readable pseudocode from the binary to understand the authentication logic with precision, without having the original source code.

Step 2.1 – Import and analyze the binary in Ghidra

After installing Ghidra and creating a new project (Lab5_SPTI), the passguess binary was imported. Ghidra automatically detected the ELF 64-bit format and x86:LE:64:default:gcc language. The automatic analysis was accepted with default settings.

Step 2.2 – Navigate to the main function and read the pseudocode

In the Symbol Tree panel, expanding Functions and clicking main displayed the reconstructed pseudocode shown in Image 23 of the lab. The critical logic was immediately visible:

```
password_match = strcmp(user_input, "7Wtyr");
if (password_match == 0) {
   puts("[OK] Passowrd Found");
} else {
```

```
    puts("[ERROR] Password is incorrect");
}
```

Step 2.3 – Rename variables for clarity

Ghidra uses generic placeholder names such as local_118 and iVar1. By right-clicking and selecting Rename Variable (or pressing L), these were renamed to user_input, password_match, and Buffer_OverFlow to make the logic self-documenting. This step is essential in real-world audits where you may return to the analysis days later.

Step 2.4 – Confirm the password and verify it

The second argument to strcmp — the string literal '7Wtyr' — was confirmed as the correct password. Running the binary and entering this string produced the [OK] Passowrd Found message, validating the analysis.

Phase 3 – Dynamic Analysis (GDB Debugger)

Objective

Execute the binary in a controlled environment to confirm register values, breakpoint behavior, and runtime execution flow — validating all static hypotheses in real time.

Step 3.1 – Start GDB and set an initial breakpoint

```
$ gdb ./passguess
(gdb) break main
(gdb) run
```

This pauses execution at the entry point of main, giving a safe starting position for further exploration.

Step 3.2 – Disassemble main within GDB and locate strcmp

```
(gdb) disassemble main
```

This displayed the full runtime disassembly of main. The strcmp call was located at address 0x5555555551e7. A precise breakpoint was set at that address to intercept execution just before the comparison.

Step 3.3 – Inspect register values at the strcmp breakpoint

```
(gdb) x/s $rdi   → 0x7fffffdc10: "hello"
(gdb) x/s $rsi   → 0x555555556004: "7Wtyr"
```

In the x86-64 calling convention, the first argument to a function is passed in $rdi and the second in $rsi. This confirmed that $rdi held the user's input ('hello') and $rsi held the actual password ('7Wtyr') — exactly as the static analysis had predicted.

Step 3.4 – Inspect the return value of strcmp

```
(gdb) stepi
(gdb) finish
(gdb) info registers rax   →  rax = 0x31  (49)
```

Since 'hello' does not equal '7Wtyr', strcmp returned a non-zero value (0x31). The test %eax, %eax instruction then set the zero flag to 0, causing the jne instruction to jump to the error block — exactly as the static CFG had predicted.

Phase 4 – Visual Flow Analysis (Cutter / Ghidra Function Graph)

Objective

Generate and interpret a visual Control Flow Graph (CFG) to confirm all decision nodes and execution paths identified in prior phases.

Step 4.1 – Generate the CFG in Cutter

The binary was opened in Cutter and the Graph → Code Flow option was selected. The resulting graph (Images 37–45 of the lab) showed: the entry block (main) leading to initialization, the scanf input-reading block, the strcmp comparison block, the critical decision node at address 001011ec where the jne branch splits execution, the success path (left branch, green) leading to [OK] Passowrd Found, and the error path (right branch, orange) leading to [ERROR] Password is incorrect, plus the stack canary check before function exit.

Ethics: Under what circumstances is it legal to reverse engineer commercial software? What precautions should you take?

3.1 Legal Framework

The legality of reverse engineering varies by jurisdiction and context. However, several widely recognized circumstances make it legally permissible:

Educational and Research Purposes

Reverse engineering carried out for educational purposes. The binary analyzed (passguess) was obtained from crackmes.one, a platform specifically designed for the practice of reverse engineering in a legal and controlled context. Crackmes are published by their authors precisely to be analyzed, granting an implicit license to reverse engineer them.

Interoperability

In many jurisdictions, including the European Union (Software Directive 2009/24/EC) and the United States under certain interpretations of the DMCA, reverse engineering is permitted to achieve interoperability between software systems — for example, to make a third-party tool compatible with a proprietary platform — provided no more information than necessary is extracted.

Security Research and Vulnerability Disclosure

Identifying and reporting vulnerabilities in software is considered a legitimate reason to reverse engineer a binary. This lab demonstrated precisely such a finding: the passguess binary stores its password in plaintext with no obfuscation, which constitutes a critical security flaw. Ethical disclosure would involve notifying the author rather than exploiting the vulnerability.

Own Software or Legally Purchased Software Without Copy Protection Circumvention

Analyzing software you own — whether you wrote it or purchased it — is generally legal. However, circumventing technological protection measures (DRM, encryption, anti-tamper systems) to do so may violate laws such as the DMCA Section 1201 in the United States or equivalent provisions in other countries.

3.2 Circumstances Where Reverse Engineering is NOT Legal

•Reverse engineering commercial software to create competing products or to extract trade secrets.

•Circumventing DRM or anti-cheat systems in violation of End User License Agreements (EULAs) or applicable law.

•Distributing cracked versions of software or any derived work that violates the original license.

•Reverse engineering software owned by someone else without authorization or a legitimate purpose recognized by law.

3.3 Precautions to Take

Before the Analysis

•Read and understand the EULA and any applicable terms of service. Many EULAs explicitly prohibit reverse engineering — even if such clauses may not always be legally enforceable, violating them can expose you to civil liability.

•Use purpose-built legal platforms such as crackmes.one, HackTheBox, or TryHackMe that provide binaries specifically authorized for analysis.

•Work in an isolated environment (virtual machine or sandbox) to prevent accidental execution of potentially malicious code affecting production systems.

During the Analysis

•Limit the scope of extraction to what is strictly necessary for the stated purpose.

•Document every step of the process (as done in this report) to demonstrate the legitimate and transparent intent of the analysis.

•Never test findings on systems you do not own or have explicit written permission to test.

After the Analysis

•Apply responsible disclosure: if a security vulnerability is found in real commercial software, notify the vendor privately before making any public disclosure — following industry-standard coordinated disclosure timelines

•Do not distribute the extracted information (e.g., hardcoded credentials, internal algorithms) in ways that could enable harm to third parties.

•Retain all documentation proving the educational or research intent of the work, in case the legitimacy of the analysis is ever questioned.